

Student Registration Number

Souleymane Sow
Bah Fatoumata
Angelah Kgato Gaolatlhe
George Ayine Ayinbangre
Djeukeng Dongmo Marie Patience
(Group 2)
African Institute for Mathematical Sciences (AIMS), Senegal

Supervised by: Yaé Ulrich Gaba
AI.Technipreneurs, Benin

October 26, 2025



AIMS

**African Institute for
Mathematical Sciences
SENEGAL**

Contents

1	Introduction	1
1.1	Background	1
1.2	Objectives	1
1.3	System Conditions	1
1.4	Program Specifications	2
2	Methodology	3
3	Description of the Functions	4
3.1	clean name function	4
3.2	extract consonants function	4
3.3	extract letters function	4
3.4	generate registration number function	5
3.5	check user input function	5
3.6	show identical prefixes	5
3.7	Main function	6
4	Conclusion	7
	References	8
A		9

1. Introduction

1.1 Background

Student registration numbers are unique numbers given to each student in an institution to identify and manage student data [1]. At AIMS Senegal, these registration numbers are formed by concatenating the first 3 consonants of the student's last name, the first 3 letters of their first name and the suffix 001. This process of assigning students registration numbers is done manually and this method is susceptible to human error and data duplication, especially if the number of students registered is high. Therefore, they require a more efficient way of registering its students that encapsulates the specific registration rules of AIMS Senegal.

1.2 Objectives

The aim is to develop a program that:

- Allows user to input each student's first and last name.
- Generates a unique registration number according to the predefined rules.
- Prevents duplicate registration
- Prints out a list of registered students and their registration numbers.

1.3 System Conditions

To maintain the validity and consistency of inputs:

1. The program must be case-sensitive.
2. Accented characters should be converted into regular characters.
3. Spaces and hyphens must be removed before processing.
4. Invalid inputs (e.g. numbers or symbols) should trigger an error message.

These constraints ensure that the program works with standardized and clean data.

1.4 Program Specifications

To generate each registration number, the program must follow the following rules:

1. Extract the first three consonants from the student's last name in uppercase, ignoring spaces and hyphens. The program should also fill the missing characters with 'X' in the case where there are less than three consonants in the last name.
2. Extract the first three letters of the first name in uppercase.
3. Add three digits (001, 002, etc.) at the end to uniquely identify each student with the same prefix.

2. Methodology

To approach this problem, we are going to write numerous functions that will perform each task separately. The first function called **clean name** will allow our program to work regardless of the case used for the input, replace all the special characters with 'normal' ones and ignore spaces and hyphens when performing the next task. The next function called **extract consonants** will get the first three consonants of the last name and convert them to capital letters. The same function will 'pad' any missing consonants from the last name with 'X' in case there are less than three consonants in the last name. In addition, there will be a function called **extract letters** that will get the first 3 letters of the first name and convert them to capital letters. The next function shall track the number of registered students who have already registered with the same prefix and assign each student a unique number. Furthermore, we shall have a function named **generate registration number** that shall ensure that the names entered contains no other characters other than alphabetic characters, spaces or hyphens. The function will also append the three consonants from the last name, the three first letters of the first name and assign unique three numbers. We shall also have the **show identical prefixes** function which will keep track of registered students with the same prefix. Lastly, we shall create a main code that will call all these functions and complete the task.

Data Flow

1. **Inputs:** (first names, last name)
2. **Validation:** Check for invalid characters.
3. **Normalization:** Convert inputs to uppercase, strip non-alphanumeric characters (spaces and hyphens), replace accented characters with normal characters.
4. **Extraction:** Generate first three consonants from last name and first three letters from first name.
5. **Sequence Update:** Use the combined consonants from last name & first three letters from first name as a key in a dictionary to get and increment the counter.
6. **Output:** Concatenate consonants from last name, first three letters from first name, and the formatted counter into the final registration number.

3. Description of the Functions

3.1 clean name function

The purpose of this function is to standardize the student's first and last names before processing. The function does this by removing all spaces and hyphens, converting all letters to uppercase, and replacing accented characters with their normal equivalents. This ensure that all names are in a uniform format, avoiding mismatches and processing errors.

```
In [10]: prenom = " éfua fatou-bintou"
nom = "Ba"
prenom = cleannome(prenom)
nom = cleannome(nom)

print(prenom)
print(nom)

EFUAFATOUBINTOU
BA
```

In the above image, a student's first names and last names were entered as 'éfua fatou-bintou Ba'. The clean name function changed the 'é' to 'e', removed all spaces and the hyphen from the firstnames and changed the letters to upper case hence 'EFUAFATOUBINTOU'. It also capitalized the last name 'Ba' to 'BA'.

3.2 extract consonants function

The function extracts the first three consonants from the last name. It does so by identifying all consonants in the last name, extracting the first three consonants and pad the missing ones with 'X' if the name contains less than three consonants, pad the missing ones with 'X'.

```
In [12]: extract_consonant(nom)

Out[12]: 'BXX'
```

The figure above shows how the extract consonants function works. The last name 'BA' has only one consonant. So the function padded it with two 'X' to 'BXX'.

3.3 extract letters function

Extracts the first three letters from the first name. In the case that the firstname has less than 3 characters, the function will also pads the missing letters with 'X'.

```
In [13]: extract_letter(prenom)

Out[13]: 'EFU'
```

In the figure above, the function `extract letter` extracts the first three letters of the first name 'EFUAFATOUBINTOU' which are 'EFU'

3.4 generate registration number function

Combines all previous results and produces the final registration number. It calls the `clean name`, `extract consonants`, and `extract letters` functions and combines the first three consonants and first three letters. The function then adds a three-digit number to ensure uniqueness among students with identical prefixes.

3.5 check user input function

Validates the user input before processing. The function ensures that only letters, spaces, and hyphens are allowed in the input. Should the input contain anything besides the mentioned characters, an error message will be displayed and a request for a new input will be made. This prevents invalid or corrupted data from being processed.

3.6 show identical prefixes

This function groups students by their matricule prefixes (excluding the last 3 characters), identifies and displays all students who share identical prefixes to show students who may have potentially conflicting registration numbers.

3.7 Main function

The main function calls all other functions to perform the complete workflow. It also displays the list of registered students with their numbers. In addition, it produces a list of students with the same prefix so that user can keep track of similar registration number.

```
enter the lastname of the student : Ba
enter the first name of the student : éfua fatou-bintou

The registration number is : BXXEFU001

Do you want to add another student ? (y/n) : y

enter the lastname of the student : Ayinbangre
enter the first name of the student : George

The registration number is : YNBGE0001

Do you want to add another student ? (y/n) : y

enter the lastname of the student : Ayinbangre
enter the first name of the student : Georgia

The registration number is : YNBGE0002

Do you want to add another student ? (y/n) : n

=== The full list of the students is ===
Ba éfua fatou-bintou ---> BXXEFU001
Ayinbangre George ---> YNBGE0001
Ayinbangre Georgia ---> YNBGE0002

=== Students with identical prefixes ===

Prefix 'YNBGE0' is shared by:
---> Ayinbangre George (YNBGE0001)
---> Ayinbangre Georgia (YNBGE0002)
```

The image above shows an output after running the main function. Two of the students happen to be twins and are both enrolled at AIMS Senegal so their prefixes are exactly identical. The combined functions differentiated the two students by assigning 001 to George who registered first and 002 to Georgia who registered after George. The function then printed out a list of the three students who registered and the list of students who have the same prefix.

4. Conclusion

The AIMS-Senegal Student Registration Automation System has been successfully developed and validated. It provides a reliable, scalable, and rules-based solution for generating unique student identifiers, fully meeting the project's objectives. However, like any other program, it has its limitations.

Limitations

- **Volatile Storage:** The registration tracker is in-memory. If the program is restarted, the sequence counter resets. It cannot persist across different run sessions.
- **Scalability:** The in-memory dictionary is suitable for a single session but would need to be replaced by a database for a large-scale, permanent system.
- **Name Ambiguity:** Students with very similar names will have very similar registration numbers, which might not be desirable.

Recommendations

- **Persistent Storage:** Integrate with a simple database (e.g., SQLite) or a file-based system to store the `registration_tracker` state permanently.
- **Graphical User Interface (GUI):** Develop a simple GUI using Tkinter or a web interface using Flask/Django for easier use by administrative staff.
- **Batch Processing:** Modify the program to accept an input file like CSV containing a list of students and output a file with all generated numbers.
- **Enhanced Uniqueness Check:** Create a permanent record system that checks for a newly registered student from the master student database even after restarting the program to avoid duplicates.

References

- [1] Matriculation number. Munich Business School, <https://www.munich-business-school.de/en/l/university-glossary/matriculation-number>, Accessed October 2025.

AppendixA.

'cleannname' function

```
def cleannname(nomprenom):
    """
    Cleans the first and last name"""
    nomprenom = nomprenom.strip().replace(" ", "").replace("-", "")
    nomprenom = nomprenom.upper()

    replacements = {
        "À": "A", "Â": "A", "Ä": "A", "Á": "A", "Ã": "A", "Å": "A",
        "É": "E", "Ê": "E", "Ê": "E", "Ë": "E",
        "Í": "I", "Î": "I", "Ï": "I", "Ì": "I",
        "Ô": "O", "Ö": "O", "Ø": "O", "Ó": "O", "Õ": "O",
        "Û": "U", "Û": "U", "Ü": "U", "Ú": "U",
        "Ç": "C", "Ñ": "N"
    }

    for accent, normal in replacements.items():
        nomprenom = nomprenom.replace(accent, normal)

    return nomprenom
```

'extract consonants' function

```
def extract_consonant(lastname):
    consonant = ""
    for letter in lastname:
        if letter in "BCDFGHJKLMNPQRSTVWXYZ":
            consonant += letter
    while len(consonant) < 3:
        consonant += "X"
    return consonant[:3]
```

'extract letter' function

```
def extract_letter(firstname):
    firstname = cleannname(firstname)
    while len(firstname) < 3:
        firstname += "X"
    return firstname[:3]

gorde=extract_letter(firstname)
print(gorde)
```

'generate registration number' function

```
def generate_registration_number(nom, prenom, register):
    lastname = cleannname(nom)
    firstname = cleannname(prenom)

    for_lastname = extract_consonant(lastname)
    for_firstname = extract_letter(firstname)

    prefix = for_lastname + for_firstname

    i=0
    register[i] = register.get(i, 0) + 1

    number = str(register[i]).zfill(3)
    return prefix + number
```

'check user input' function

```
def Check_User_Input(fullname):
    while True:
        try:
            if fullname.replace(" ", "").replace("-", "").isalpha():
                #print(f" great: valid input '{user_input}')
                return fullname
            else:
                raise ValueError(" invalid Input; please enter a valid name ")
        except ValueError as e:
            print(e)
            fullname = input("try again : ")
```

'show identical prefixes' function

```
def show_identical_prefixes(students):

    prefix_groups = {}

    for name, matricule in students.items():
        prefix = matricule[:3]
        if prefix not in prefix_groups:
            prefix_groups[prefix] = []
        prefix_groups[prefix].append(name)

    print("\n=== Students with identical prefixes ===")
    found = False
    for prefix, names in prefix_groups.items():
        if len(names) > 1:
            found = True
            print(f"\nPrefix '{prefix}' is shared by:")
            for n in names:
                print(f"    ---> {n} ({students[n]})")

    if not found:
        print("No identical prefixes found.")
```

Main function

```
if __name__ == "__main__":
    register = {} #prefix

    students = {} #new dictionary to save students' information

    while True:
        lastname = input("enter the lastname of the student : ")
        lastnameS = Check_User_Input(lastname)
        firstname = input("enter the first name of the student : ")
        firstnameS = Check_User_Input(firstname)

        # Generation of the registration_number
        registration_number = generate_registration_number(lastnameS, firstnameS, register)
        print("\n The registration number is :", registration_number)

        # Dictionary entry
        students[f"{lastnameS} {firstnameS}"] = registration_number

        other = input("\nDo you want to add another student ? (y/n) : ").strip().lower()
        if other != "y":
            break
        print("")

    # Print the final dictionary
    print("\n=== The full list of the students is ===")
    for identity, mat in students.items():
        print(f"{identity} ---> {mat}")

    show_identical_prefixes(students)
```